Practice Questions 1 - Programming in Python

1. Which of the following is allowed in Python?

   a. `x+y=2`

   b. `x*x=2`

   c. `2=x`

   d. `xy=2`

   e. None of these

2. What is the value of the variable `u` from the code below:

```
once = 'umbr'
repeat = 'ella'
u = once + (repeat + ' ')*4
```

   a. umbrella

   b. umbrellaellaellaella

   c. umbrella ella ella ella

   d. umbrella4

3. You run the code below:

```
us_gold = 46
uk_gold = 27
rus_gold = 1
total_gold = us_gold + uk_gold + rus_gold
print(total_gold)
rus_gold += 100
print(total_gold)
```

   What is the output?

   a. 74

   74

   b. 74 74

c. 74 174

d. 174 74

4. Which of the following is/are valid variable names?

a. For b. While c. In d. 123In

5. What is printed when you enter "Right" in the input entry?

```
n = input("Type something: ")
while n == "right":
    print("You are Correct")
else:
    print("Incorrect Answer")
```

a. Incorrect answer **b. Incorrect Answer** c. You are correct d. You are Correct

6. What does the code below print?

```
s = "6.00 is 6.0001 and 6.0002"
new_str = ""
new_str += s[-1]
new_str += s[0]
new_str += s[4::30]
new_str += s[13:10:-1]
print(new_str)
```

a. 26000 b. 26100 **c. 26 100** d. Error e. 6.00 is 6.0001 and 6.0002

7. How many times will "Gate DA" be printed?

```
s1 = "mit u rock"
s2 = "i rule mit"
if len(s1) == len(s2):
    for char1 in s1:
```

```
        for char2 in s2:
            if char1 == char2:
                print("Gate DA")
                break
```

a. 4 b. 6 **c. 7** d. 8 e. None of these

8.
```
L = ["life", "answer", 42, 0]
for v in L:
    if v == 0:
        L[v] = "Venky"
    elif v == 42:
        L[3] = "Viky"
        L[1] = 0
print(L)
```

What is the output of the code?

a. ['life', 'answer', 42, 'Viky']

b. ['Venky', 0, 42, 'Viky']

c. ['life', 'Venky', 'answer', 'Viky']

d. ['life', 0, 42, 'Viky']

9. What is the value of L3 on executing the operations below:

```
L1 = ['re']
L2 = ['mi']
L3 = ['do']
L4 = L1 + L2
L3.extend(L4)
L3.sort()
del L3[0]
L3.append(['fa', 'la'])
```

a. ['re', 'mi', ['fa', 'la']]

b. ['mi', 're', 'fa', 'la']

c. ['mi', 're', ['fa', 'la']]

d. None

10. What will the output be?

```
L = 3
for i in range(len(L)):
    print(i)
```

a. 0 1 2

b. 0

1

2

c. Error

d. 0

11.
```
L1 = ['bacon', 'eggs']
L2 = ['toast', 'jam']
brunch = L1
L1.append('juice')
brunch.extend(L2)
```

What is the value of `brunch` ?

a. `['bacon', 'eggs', 'toast', 'jam']`

b. `['bacon', 'eggs', ['toast', 'jam']]`

c. `['bacon', eggs, 'juice', 'toast', 'jam']`

d. `['bacon', eggs, 'juice', ['toast', 'jam']]`

12. What is the output of `any([2 > 8, 4 > 2, 1 > 2])` ?

a. `True` b. `False` c. `4 > 2` d. None

13. What is the output of `any(1, 0.2, 3)` ?

a. `True` b. `False` **c. Error, any takes iterables as a single argument.** d. None

14. What are the outputs of `sum(2, 4, 6)` and `sum([1, 2, 3])`?

    a. 12, Error b. 6, Error c. 12, 6 **d. Error, 6**

    The `sum()` function takes either an iterable or 2 arguments.

15. What is the output of `min(max(False, -3, -4), 2, 7)`?

    a. 0 b. -4 **c. False** d. None

16. What is the output of `list(enumerate([2, 3]))`?

    a. Error b. [(1, 2), (2, 3)] c. [(2, 3)] **d. [(0, 2), (1,3)]**

17. The function `complex(2-3j)` is valid but the function `complex('2-3j')` is invalid. True or false?

    a. True b. False, both are invalid **c. False, both are valid** d. False, the first is invalid, the second is valid

18. The function `complex(2-3j)` is valid but the function `complex('2 - 3j')` is invalid. True or false?

    a. True b. False, both are invalid c. False, both are valid d. False, the first is invalid, the second is valid

    `complex(2+3j)` and `complex(2 + 3j )` both are valid.

    `complex('2+3j')` is valid, but `complex('2 + 3j')` isn't.

    `complex(3j+2)` is valid, but `complex('3j+2')` isn't.

19. What is the output of:

    a. `float('1e-003')` : **0.001**

    b. `float(2e+003)` : **2000.0**

    c. `float()` : **0.0**

    d. `complex()` : **0j**

20. What is the output of `float(' -12345\n')` ?

    a. `-12345.0` **b. `-12345.0`** c. `Error` d. `-12345.00000...`

21. What will be the value of `Virat` in the following code?

```
Virat = Rohit
Virat += Sachin
print(Virat)
```

    a. RohitSachin b. SyntaxError c. TypeError d. ValueError **e. NameError**

22. What does `~~~~~~5` evaluate?

    a. +5 b. -11 c. +11 d. -5 e. Error

    ~(n) = -(n+1)

    ~(5) = -6

    ~(-6) = -(-5) = 5

    and so on

23. Which of the following is incorrect?

    a. `x = 0b101` b. `x = 0x4f5` c. `x = 19023` **d. `x = 03964`** e. None of these

    `x = 0b101` is base 2, x = 5 (binary 101)

`x = 0x4f5` is base 16, x = 1269 (hexadecimal 4f5)

`x = 19023` is just base 10.

`x = 03964` is incorrect as leading zeros are not allowed.

For octal numbers, `0o` would be used as the leading bits. Like `x = 0o3964` .

24. What is the output of:

```
x = ['ab', 'cd']
for i in x:
    i.upper()
print(x)
```

a. ['ab', 'cd'] b. ['AB', 'CD'] c. [None, None]

25. What is the output of:

```
x = ['ab', 'cd']
for i in x:
    x.append(i.upper())
print(x)
```

a. ['ab', 'cd'] b. ['AB', 'CD'] c. ['ab', 'cd', 'AB', 'CD'] d. Error **e. The program goes into an ∞ loop**

26. What is the output of:

```
x = ['ab', 'cd']
for i in range(len(x)):
    x.append(x[i].upper())
print(x)
```

a. ['ab', 'cd'] b. ['AB', 'CD'] **c. ['ab', 'cd', 'AB', 'CD']** d. Error e. The program goes into an ∞ loop

27.
```
i = 1
while True:
    if i % 3 == 0:
        break
    print(i)
    i += 1
```

a. 1

2

b. 1 2

c. 1 2 3

d. 1

2

3

28.
```
True = False
while True:
    print('hello there')
```

What is the output?

a. KeywordError **b. SyntaxError** c. NameError d. No output

29.
```
i = 5
while True:
    if i % 0o11 == 0:
        break
    print(i)
    i += 1
```

What is the output of the code?

a.

5

6

7

8

b. 5 6 7 8

c. 5 6

d. 5 6 7 8 9 10 11

30.
```
i = 5
while True:
    if i % 0o9 == 0:
        break
    print(i)
```
What is the output?

a. 5 6 7 8 b. 5 6 7 8 9 c. 5 6 7 8 9 10 **d. None**

SyntaxError: invalid digit '9' in octal literal

31. What is the output of the following?

```
 i = 1
while True:
    if i % 2 == 0:
        break
    print(i)
    i += 2
```
a. 1 b. 1 2 c. 1 2 3 4 5 6... **d. 1 3 5 7 9...**

32. What is the output of the following?

```
 i = 1
while False:
    if i % 2 == 0:
        break
    print(i)
    i += 2
```

a. 1 b. 1 3 5 7 c. 1 2 3 4 **d. None**

33. What is the output?

```
i = 0
while i < 5:
    print(i)
    i += 1
    if i == 3:
        break
else:
    print(0)
```

a. 0 1 2 3 b. 0 1 2 0 **c. 0 1 2** d. None

34. What is the output?

```
i = 0
while i < 3:
    print(i)
    i += 1
else:
    print(0)
```

a. 0 1 2 3 **b. 0 1 2 0** c. 0 1 2 d. None

A `break` skips the `else` statement for the loop as well.

35. What is the output?

```
x = 'abcdef'
while i in x:
    print(i, end=' ')
```

a. a b c d e f b. abcdef c. i i i i i ... **d. None**

This would work in the `for` loop, not in the `while` loop.

36. What is the output?

```
x = 'abcdef'
i = 'i'
while i in x:
    print(i, end=' ')
```

a. i i i i ... b. a b c d e f c. i i i i i i d. iiiiii **e. No output**

37. What is the output?

```
x = 'abcd'
for i in range(len(x)):
    print(i.upper(), end = ' ')
print(x)
```

a. A B C D b. A B C D ABCD c. SyntaxError d. A B C D abcd **e. AttributeError**

38. What is the output?

```
x = 'abcd'
for i in x:
    print(i.upper(), end=' ')
print(x)
```

a. A B C D b. A B C D ABCD c. SyntaxError d. A B C D abcd e. AttributeError

39. What is the output?

```
x = 'abcd'
for i in range(len(x)):
    x = 'a'
    print(x, end=' ')
```

a. a a a a b. abcd abcd abcd abcd c. a d. None

40. What is the output?

```
x = 'abcd'
for i in range(len(x)):
    print(x, end=' ')
    x = 'a'
```

a. abcd abcd abcd abcd b. a a a a **c. abcd a a a** d. abcd aaaa

41. What is the output?

```
x = 123
for i in x:
    print(i)
```

a. 123 b. SyntaxError: 'int' object is not iterable **c. TypeError: 'int' object is not iterable**

42. What is the output?

```
d = {0: 'a', 1: 'b', 2: 'c'}
for i in d:
    print(i, end=' ')
```

a. a b c b. 0 a 1 b 2 c **c. 0 1 2**

43. What is the output?

```
str1 = 'hello'
str2 = ','
str3 = 'world'
str1[-1:]
```

a. olleh b. h c. hello **d. o** e. None

44. What is the output?

```
str1 = 'hello'
str2 = ','
str3 = 'world'
str1[-1::-1]
```

a. olleh b. h c. hello d. o e. None

45. Which of the following would cause an error?

a. `'"Once upon a time...", she said'`

b. `"He said, 'yes'"`

c. `"3\"`

d. None

Note: `len('3\"')` would be 2.

46. What is the output?

```
print('new''line')
```

a. Error b. new line **c. newline** d. None

47. What is the output?

```
print('new','line')
```

a. Error **b. new line** c. newline d. None

48. What is the output?

```
print(r"\nhello\nworld")
```

a.

```
hello
world
```

b.

```
\nhello\nworld
```

c.

```
helloworld
```

49. What is the output?

```
0xA + 0xD + 0xC
```

a. 0xA0xD0xC **b. 33** c. Error

50. What is the output?

```
names1 = ['Amir', 'Bear', 'Charlton', 'Daman']
names2 = names1
names3 = names1[:]
names2[0] = 'Alice'
names3[1] = 'Bob'
sum = 0
for ls in (names1, names2, names3):
    if ls[0] == 'Alice':
        sum += 1
    if ls[1] == 'Bob':
        sum += 10
print(sum)
```

a. 12 b. 10 c. 1 d. 22

51. What is the output?

```
names1 = ['venky', 'viky', 'rbr']
names[-1][-2]
```

a. viky b. rbr **c. b**

52. The `clear()` method is used to delete a dictionary. True or false?

a. True b. False c. Cannot be determined

53. Consider a tuple `t1` . Which of these is an invalid operation?

a. `t1 + (12, 24)` **b. `t1 + (24)`** c. `t1 + tuple([24])` **d. `t1 + [12, 24]`**

Tuples can only be concatenated with tuples.

54. Which among these are valid ways to create a dictionary?

a. d = {} b. d = [] c. d = dict{} d. d = ()

55. Which of these is/are invalid?

a. `a_b_c = 1,000,000`

b. `abc = 1000,000,000`

c. `a b c = 1000,000,000`

d. `a, b, c = 1000,000,000`

e. `a, b, c = (1000, 2000, 3000)`

56. Which of these is/are valid?

a. `a, b = 1, 2, 3`

b. `a, b, c = 1, 2`

c. `a, *b = 1, 2, 3`

d. `*a, b = 1, 2, 3`

e. `*a, *b, c = 1, 2, 3, 4, 5`

Only c and d are valid. Either the exact # of values to unpack should be provided, or a starred expression should be used to unpack values as a list into the starred variable. Multiple starred expressions are not allowed.

57.
```
t1 = 1, 2.0, (False, False), 12.5, [False], {12, 10, 12.0}
print(len(t1), len(t1[5]))
```
What is the output?

6, 2

58. There's no 58. Fuck off.

End.